

# How to make AI better at product 🎨

Exploring ideas based on experiments and research.



LUCA ROSSI

JUN 17, 2026



48



Article voiceover

0:00

-13:16

Hey, [Luca](#) here! Welcome to a ✨ **monthly free essay** ✨ from Refactoring!

Every week I write an article about how to ship fast and make good software, I interview a world-class tech leader. I also build and maintain [Tolaria](#) in the open and publishing my workflows and learnings here.

Last week, Claude Code's Boris Cherny got trending on X for saying this at the Anthropic conference:

*I don't prompt Claude anymore. I have loops running that prompt Claude and figure out what to do. **My job is to write loops.** And this is the transition we're going to see for the rest of the year*

So, "my job is to write loops."

Peter Steinberger (creator of OpenClaw) and several others picked up on it and agreed 🙌



**Peter Steinberger** 🦀 🔵 📺  
@steipete



Here's your monthly reminder that you shouldn't be prompting coding agents anymore.

You should be designing loops that prompt your agents.

8:58 PM · Jun 7, 2026 · 4.4M Views

But before you roll your eyes and think here it goes another *fad*, new buzzword *loop engineering*, and so on, I'll go out and say that I like this direction.

*Loop engineering* looks strictly better than *prompt engineering* because it forces us into **systems thinking**, as opposed to just focusing on what's needed to deliver the current task. A loop is supposed to run indefinitely, so the focus shifts to reliability, sustainability, and how to make sure things stay good — vs *diverge* and go to s

But how do you ensure that?

I wrote about this several times lately, including [last week](#), where I covered how to steer AI work with the 3Gs: *guides*, *gates*, and *guards*. These are not super new concepts: they build on what other smart people are doing, and they are my attempt to contribute to this global conversation.

But a problem I find in this conversation (which I am guilty of) is that it largely **focuses on the tech side** of things, while treating other disciplines as second-class citizens.

Namely, **product**.

When you hear people saying that software dev is now a factory, and the bottleneck is your product taste, review, validation, etc, what they are implying is that they have no idea about how to scale those things. In the AI race, product is behind engineering — big time!

The last time we discussed this on Refactoring has been when we published [The State of Product Development](#), where we saw that only 9% of teams used AI to

product with specs (yay), vs 90%+ for coding. Since then I released Tolaria, started doing a lot of product dev myself, and tried a lot of different solutions and workflows to scale myself properly, both in tech and product.

So today I want to focus on product, taking pages from what works for *coding*, exploring how we can make the same progress there.

To discuss this, I am also bringing in again [Doug Peete](#) from [Atono](#), who has a privileged vantage point by working with thousands of product teams, and is [working actively](#) to fix this. We also talked a lot about AI & product in our [recent podcast interview](#).

So here is the agenda:

-  **Product is harder than coding** — an uncomfortable take we need to come to terms with.
-  **Towards Product ADRs** — building on a successful tech practice.
-  **Glossary** — a possible missing abstraction in how we develop product.
-  **Product process** — how to put everything together and work better with it.

Let's dive in!

*Disclaimer: I am a fan of what [Atono](#) is building and I am grateful to have it as a partner on this piece. You should check it out!*

*However, I will only write my unbiased opinion about the practices and tools covered, Atono included.*



## Why product is harder than coding

Product is harder than coding, at least conceptually. I know this take will get many engineers angry, but think about it. For a product choice to be good, there is a lot

be taken into account:

- 🚀 **Strategy** — do customers want this? Do the *right* customers want this? is this discoverable? Is this making things unnecessarily complex? How do fit our business strategy?
- 🔍 **Tactics** — where do we put this? what's the right UX? does it need a shortcut? should this live in the right-click menu? ...

A lot of these choices depend on aspects that we are not good at *codifying* into rules. It's what we call *judgment* and *taste*.

Software engineering, on the other hand, is *structurally* easier, for two reasons

- It's *downstream* of product direction, so it has a narrower design space.
- Its *deterministic* nature allows it to be tested for correctness and non-functional qualities (performance, cohesion/coupling, complexity) in a more reliable way than product.

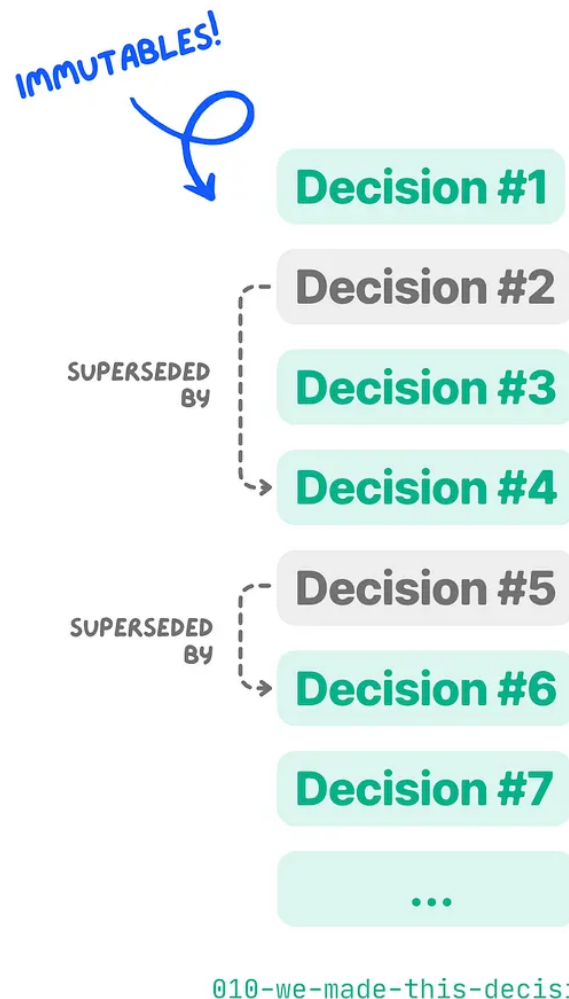
As a result, **gates for coding agents** are effective at enforcing good code, even they have relatively little context about product. Static analysis, TDD, code health tooling, are all devices that help AI agents write *good enough* code—granted, not code that the most talented engineer on earth would write—but still passable in many situations.

So how do we make product a bit more like coding? To answer this question, I will take a page from the *least deterministic* part of my coding harness: ADRs.

## **ADRs**

As I have written many times, I instruct the AI to create [ADRs](#) to store **meaningful tech decisions**. Each ADR has to include why that decision was made, what options were considered (and eventually discarded), and take care of *superseding* old ADRs about the same thing.

ADRs have been, so far, surprisingly successful. As I am writing this, Tolaria has ADRs, and I see the AI using them all the time. It refers to them in specs, makes decisions that stay true to what exists, and reliably supersedes what's not relevant anymore.



So, as of today, for most features I am comfortable not giving a lot of (tech) design input, because I know AI will pick it up from past decisions. I still review new AI decisions but they are 95% good — I do it more for my personal curiosity than because it's a good use of my time.

The ADR workflow is, interestingly, **completely non-deterministic**. It's a way to *inject* good judgment into the AI not by creating rules, but by repeatedly showing how we have done things in the past.

This is why I am hopeful about doing something similar to product 🙌

## Towards Product ADRs

If ADRs are for tech work, what's the equivalent for product? Mimicking ADRs, a *Product Decision Record* might record:

-  **Intent** — what the user needs to accomplish.
-  **Design** — how we decided to do it.
-  **Tradeoffs** — alternatives that have been discarded and why.

If we roughly agree on this, the next question would be: what does actually describe a PDR? To me, it's any **product decision we want to remember** in case we meet a similar problem in the future. Anything genuinely *new* that we changed/added to the product, as opposed to reusing a proven pattern that we already understand well.

E.g. In Tolaria, I wouldn't want to write a PDR to remember that we added "copy link" as a right-click action to the current note — because there isn't any novel or interesting decision being made there.

Do PDRs map squarely to well-known product artifacts, like PRDs, Epics, or Stories? I don't know about that either. Those other devices primarily exist to communicate *what* needs to be done. It's great when they include more about the *why*, but it's not clear to me whether e.g. a full PRD or user story should be stored as-is and well as a PDR. Maybe!

The other reason why I tend to believe PDRs deserve a life on their own is that relevant *decisions* are largely **independent of the size of the work**. There might be a large Epic (e.g. integrating a new AI agent assistant into Tolaria) that doesn't bring a meaningful new product direction, and a small story that instead introduces a new and interesting pattern.

Doug is experimenting with PDRs at Atono, calling them [design decisions](#) and attaching them directly to the stories they relate to — not as a separate doc for

This doesn't resolve the question of where a PDR lives when the decision is bigger than any single story, but looks good on the smaller end of the spectrum: when decision is scoped to one piece of work, attaching it directly might hold up well in practice.

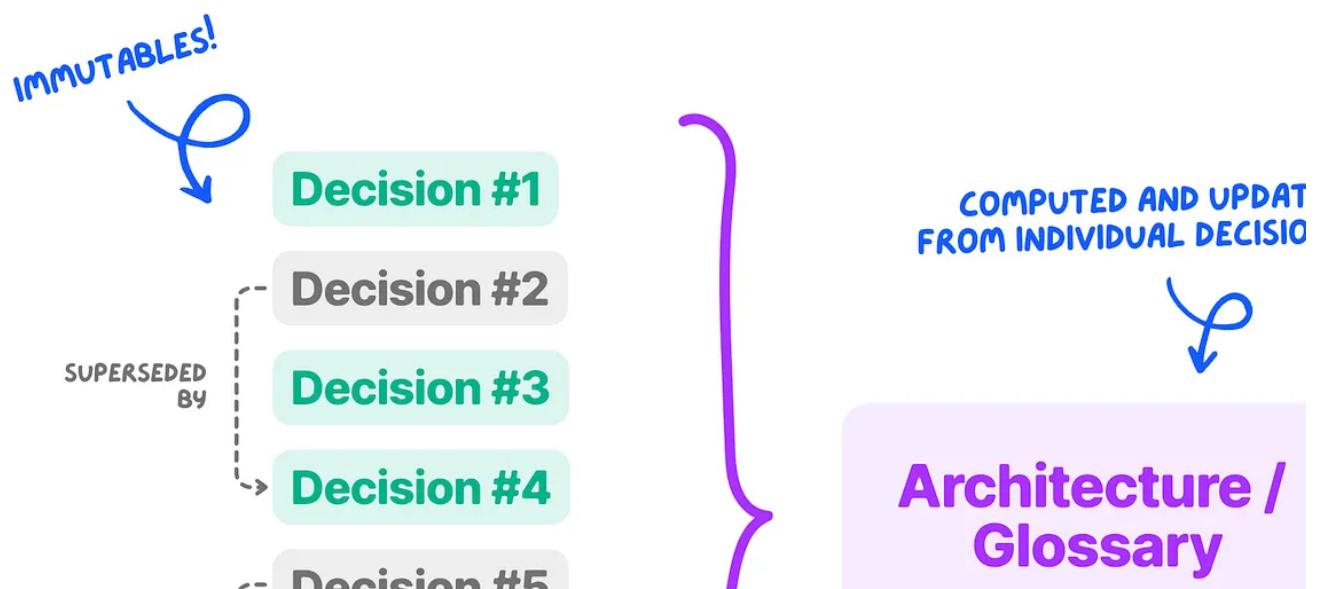
So, in general, even if there is a lot that we don't know about their *shape*, we can agree that it's valuable to record product decisions, in a similar way to what we do with architecture ones. And if we do, we should also agree that this is not enough.

## Product Glossary

Other than ADRs, I maintain other tech docs in the Tolaria repo, which are called ARCHITECTURE.md and ABSTRACTIONS.md. Without getting into too much detail, these are **recap docs**, that summarize the current state of our tech.

These are necessary, like views in a database, to avoid the AI to navigate event hundreds of ADRs (between those active and superseded) every time, just to **reconstruct what exists**. The recap docs give the AI the 10,000ft view, while individual ADRs are used to zoom into specific aspects.

What should this look like for product? Doug believes the right concept here is of a [Glossary](#).





The Glossary should list your main product abstractions: why they exist, what are for, and how they are used. It maps your *domain language*, which in turn is shaped by your product surface.

An example from Tolaria may include concepts like:

- **The Sidebar** — which gives users ways to *filter* notes in their workspace
- **The Note List** — to make users navigate the current selection of notes
- **The Breadcrumb Bar** — to display the most important actions a user can do over the current note.

But also concepts that don't map 1:1 to single interface elements, like the **Git integration**, or the **Note organization workflow** (capture / organize / archive)

It's obvious these are not PDR candidates: they are long-lived abstractions that morph over time, and need a description that evolves with them. So, if anything are *derived* from a sum of product decisions.

Doug believes a shared glossary is the cornerstone that enables good **AI-assisted stories and PRDs** — because now the AI can ground them in your vocabulary, your past choices, and what exists. And he has data to back this up.

By talking with some of the teams we surveyed, which report [abysmal low AI adoption](#) for product specs (9%), it turns out a major cause is the amount of rework needed by generic AI specs. Making them good is just too much work, so most folks keep writing specs manually. Doug and Atono have been testing glossary-backed

stories internally and with customers, and they are seeing rework dropping from 60% to 20%.

(worth noting that 60% rework, as a baseline, is a ton. What if coding agents go things wrong 6 times out of 10? We would probably give up on them, which is what many PMs do with AI right now)

So the overall goal is to build foundational product knowledge: a map of what it means in this specific product, which is not the same thing as a list of features.

So let's say we have all of this in place, what does a new product process look like?

## Product process

Any product work starts with **capturing some intent** — what the user needs to accomplish, how it maps to the product strategy, how we know this is successful and so on.

This is 100% human (for now!) and is more about the *why* than the *what*.

Our input on the *what* can progressively shrink the more we help the AI figure out good (draft?) specs that respect the current product surface (glossary + PDRs).

Then, based on the feature, the maturity level of your process, and how greedy/safe you want it to be, you may either:

- Have the AI do a first pass, so that your judgement is informed by an actual prototype, or
- Stop and tweak the draft spec before the AI goes forward.

Then you review and iterate on implementation as long as needed, and finally ship, possibly including new PDRs and Glossary updates.

Over the next weeks I will try to incorporate some of this into the Tolaria workflow.

possibly *retrofitting* some PDRs and glossary concepts from the past. Will let y know how it works!

And that's it for today! I wish you a great week

Sincerely 🖐️

Luca

---

## Subscribe to Refactoring

Thousands of paid subscribers

Learn how to make your engineering team ship faster and better. Read weekly by 170,000+ engineers and managers

By subscribing, you agree Substack's [Terms of Use](#), and acknowledge its [Information Collection Notice](#) and [Privacy Policy](#).



48 Likes

## Discussion about this post

Comments

Restacks



Write a comment...

